

## Lab 4 – Elimination of Ambiguity, Left Recursion and Left Factoring

### Aim

To eliminate ambiguity, left recursion, and perform left factoring for a given grammar so that it becomes suitable for predictive parsing.

### Theory

#### Elimination of Ambiguity

A grammar is **ambiguous** if a string can have more than one parse tree.

Example:

$$E \rightarrow E + E \mid E * E \mid id$$

A non-ambiguous form is:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow id$$

#### Left Recursion

A grammar is **left recursive** if a non-terminal appears as the leftmost symbol of its own production.

General form:

$$A \rightarrow A\alpha \mid \beta$$

After elimination:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \varepsilon$$

#### Left Factoring

Left factoring is applied when two or more productions have a common prefix.

General form:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

After left factoring:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2$$

### Algorithm – Left Recursion Elimination

1. Start the program.
2. Read a grammar of the form  $A \rightarrow A\alpha \mid \beta$ .
3. Separate the productions using  $\mid$ .
4. Identify:
  1. recursive production beginning with A
  2. non-recursive production
5. If left recursion exists:
  1. print  $A \rightarrow \beta A'$
  2. print  $A' \rightarrow \alpha A' \mid \epsilon$
6. Otherwise print **No left recursion**.
7. Stop.

### C Program – Left Recursion Elimination

```
#include <stdio.h>
#include <string.h>
int main() {
    char grammar[100], alpha[50], beta[50];
    char lhs;

    printf("Enter grammar (Example: E->E+T|T): ");
    scanf("%s", grammar);

    lhs = grammar[0];

    char *arrow = strstr(grammar, "->");
    if (arrow == NULL) {
        printf("Invalid grammar\n");
        return 0;
    }

    char rhs[100];
    strcpy(rhs, arrow + 2);
    char *part1 = strtok(rhs, "|");
    char *part2 = strtok(NULL, "|");

    if (part1[0] == lhs) {
        strcpy(alpha, part1 + 1);
        strcpy(beta, part2);

        printf("\nGrammar after removing left recursion:\n");
        printf("%c -> %s%c\n", lhs, beta, lhs);
        printf("%c -> %s%c' | e\n", lhs, alpha, lhs);
    }
}
```

```

    }
    else if (part2 != NULL && part2[0] == lhs) {
        strcpy(alpha, part2 + 1);
        strcpy(beta, part1);

        printf("\nGrammar after removing left recursion:\n");
        printf("%c -> %s%c\n", lhs, beta, lhs);
        printf("%c' -> %s%c' | e\n", lhs, alpha, lhs);
    }
    else {
        printf("\nNo left recursion found.\n");
    }
}

return 0;
}

```

## Sample Input

E->E+T|T

## Sample Output

Grammar after removing left recursion:

E -> TE'

E' -> +TE' | e

## Algorithm – Left Factoring

1. Start the program.
2. Read grammar of the form  $A \rightarrow \alpha\beta_1 | \alpha\gamma_2$ .
3. Compare the common prefix of both productions.
4. Store the common prefix.
5. Print:

1.  $A \rightarrow \alpha A'$
2.  $A' \rightarrow \beta_1 | \beta_2$

6. Stop.

## C Program – Left Factoring

```
#include <stdio.h>
```

```
#include <string.h>
```

```

int main() {
    char grammar[100], lhs;
    char first[50], second[50];
    char prefix[50];
    int i = 0;

```

```

printf("Enter grammar (Example: A->abX|abY): ");
scanf("%s", grammar);

lhs = grammar[0];

char *arrow = strstr(grammar, "->");
if (arrow == NULL) {
    printf("Invalid grammar\n");
    return 0;
}
char rhs[100];
strcpy(rhs, arrow + 2);

strcpy(first, strtok(rhs, "|"));
strcpy(second, strtok(NULL, "|"));

while (first[i] == second[i] && first[i] != '\0') {
    prefix[i] = first[i];
    i++;
}
prefix[i] = '\0';

if (strlen(prefix) == 0) {
    printf("No left factoring required.\n");
    return 0;
}

printf("\nGrammar after left factoring:\n");
printf("%c -> %s%c\n", lhs, prefix, lhs);
printf("%c' -> %s | %s\n", lhs, first + i, second + i);

return 0;
}

```

## Sample Input

A->abX|abY

## Sample Output

Grammar after left factoring:

A -> abA'

A' -> X | Y

## Result

Thus ambiguity was studied, left recursion was eliminated, and left factoring was performed successfully for the given grammar.